

**LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 12**



Sultan Zulvi Risda

(105224034)

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA**

2026

1. Pendahuluan

Sistem Informasi Akademik (SIAKAD) merupakan infrastruktur teknologi inti pada institusi pendidikan tinggi yang mengelola siklus akademik mahasiswa, khususnya dalam prosedur pengisian Kartu Rencana Studi (KRS). Namun, sistem legacy (lama) sering kali mengalami degradasi performa yang parah hingga status crash total pada masa beban puncak akibat diakses oleh ribuan mahasiswa secara simultan. Berdasarkan hasil audit internal, ditemukan akar masalah struktural pada kode sumber, di mana seluruh beban logika bisnis bertumpu secara monolitik pada sebuah objek raksasa bernama SistemKRSManager. Keterikatan yang sangat ketat (tight coupling) ini memicu efek domino kegagalan: perubahan minor pada format presentasi visual (seperti kop surat draf PDF) berimbas langsung pada kegagalan penyimpanan basis data transaksional. Arsitektur semacam ini meningkatkan akumulasi utang teknis (technical debt) dan menghambat pemeliharaan serta ekspansi sistem ke depan.

2. Struktur Package

Berikut adalah visualisasi nyata dari struktur package yang diimplementasikan pada proyek Java ini:

src/

```
|— KRS/
|   |— KRSPdfGenerator.java
|   |— KRSValidator.java
|   └— SistemKRSManager.java
|— main/
|   └— MainApp.java
|— matakuliah/
|   |— LaboratoriumAkses.java
|   |— MataKuliah.java
|   |— MataKuliahPraktikum.java
|   └— MataKuliahTeori.java
|— repository/
|   |— CloudNoSQLKRSRepository.java
|   └— KRSDataRepository.java
└— strategy/
    └— InternationalUKTStrategy.java
```

- |— MbkmUKTStrategy.java
- |— RegulerUKTStrategy.java
- |— UKTCalculationStrategy.java

3. Penerapan SOLID Principles

- Single Responsibility Principle (SRP)

Kelas raksasa SistemKRSManager didekomposisi menjadi tiga entitas mandiri yang masing-masing hanya mengemban satu tanggung jawab spesifik:

- KRSValidator: Terfokus secara eksklusif pada validasi prasyarat akademik mata kuliah.
- KRSPdfGenerator: Bertanggung jawab penuh atas formating dokumen draf KRS ke PDF.
- SistemKRSManager: Bertindak sebagai koordinator orkestrasi proses alur pendaftaran tanpa mengetahui detail teknis cetak atau validasi data.

- Open/Closed Principle (OCP)

Untuk mengeliminasi tumpukan kendali pengkondisian logika if-else yang rapuh pada perhitungan skema UKT, diterapkan Strategy Pattern. Dibuat sebuah abstraksi bernama UKTCalculationStrategy. Ketika pihak kampus meluncurkan program baru seperti Merdeka Belajar Kampus Merdeka (MBKM), pengembang cukup menambahkan kelas konkrit baru (MbkmUKTStrategy) tanpa perlu membongkar atau mengubah baris kode kalkulasi inti yang telah stabil.

- Liskov Substitution Principle (LSP)

Pada sistem terdahulu, kelas mata kuliah Teori dipaksa mengimplementasikan metode laboratorium yang mengakibatkan dilemparnya UnsupportedOperationException. Melalui rancangan ulang, kelas MataKuliahTeori hanya berakar pada antarmuka umum MataKuliah, sedangkan metode lab dipisahkan. Hal ini menjamin bahwa seluruh subtype objek dapat saling menggantikan secara polimorfis tanpa pernah merusak alur eksekusi runtime sistem.

- Interface Segregation Principle (ISP)

Kontrak antarmuka yang gemuk dipecah menjadi beberapa modul spesifik yang ramping. Antarmuka MataKuliah hanya mengurus properti akademik dasar (kode, nama, SKS). Sementara kapabilitas operasional laboratorium diisolasi secara ketat ke dalam antarmuka terpisah bernama LaboratoriumAkses. Kelas non-praktikum tidak dipaksa untuk bergantung pada fungsionalitas yang tidak mereka gunakan.

- Dependency Inversion Principle (DIP) & Decoupling Layer (DL)
Kelas pengontrol tingkat tinggi tidak lagi melakukan instansiasi langsung terhadap koneksi database konkrit via kata kunci new. Komponen bisnis kini bergantung penuh pada abstraksi murni berupa antarmuka KRSDataRepository. Dengan teknik Dependency Injection (DI) melalui konstruktor, perpindahan medium penyimpanan dari MySQL menuju Cloud NoSQL dapat dituntaskan secara instan di kelas utama tanpa memodifikasi logika bisnis KRS sedikit pun.

4. Penjelasan Program

- Package mataKuliah

```
package matakuliah;  
  
public interface MataKuliah {  
  
    String getKode();  
  
    String getName();  
  
    int getSks();  
  
}
```

Kontrak dasar yang mendefinisikan atribut esensial dari setiap mata kuliah di universitas.

```
package matakuliah;  
  
public interface LaboratoriumAkses {  
  
    void alokasiAsistenLab();  
  
    void cekPeralatanPraktikum();  
  
}
```

Antarmuka spesifik (ISP) yang memisahkan kapabilitas operasional praktikum agar tidak mengotori kelas teori.

```
package matakuliah;  
  
public class MataKuliahTeori implements MataKuliah {  
  
    private String kode;  
  
    private String nama;  
  
    private int sks;  
  
    public MataKuliahTeori(String kode, String nama, int sks) {  
  
        this.kode = kode;  
  
        this.nama = nama;  
  
    }  
  
}
```

```

        this.sks = sks;

    }

    @Override public String getKode() { return kode; }

    @Override public String getNama() { return nama; }

    @Override public int getSks() { return sks; }

}

```

Representasi mata kuliah non-praktikum yang murni mengimplementasikan atribut dasar akademik secara aman (LSP).

```

package matakuliah;

public class MataKuliahPraktikum implements MataKuliah,
LaboratoriumAkses {

    private String kode;

    private String nama;

    private int sks;

    public MataKuliahPraktikum(String kode, String nama, int sks) {

        this.kode = kode;

        this.nama = nama;

        this.sks = sks;

    }

    @Override public String getKode() { return kode; }

    @Override public String getNama() { return nama; }

    @Override public int getSks() { return sks; }

    @Override

    public void alokasiAsistenLab() {

        System.out.println("Mengalokasikan asisten lab untuk praktikum
" + nama);

    }

    @Override

    public void cekPeralatanPraktikum() {

```

```

        System.out.println("Memeriksa kesiapan alat lab untuk praktikum
" + nama);
    }
}

```

Kelas konkrit yang mengimplementasikan karakteristik akademik sekaligus kapabilitas teknis laboratorium.

- Package Strategy

```

package strategy;

public interface UKTCalculationStrategy {

    double hitungTagihan(int totalSks);

}

```

Abstraksi strategi polimorfik untuk penentuan biaya pendidikan mahasiswa (OCP).

```

package strategy;

public class RegulerUKTStrategy implements UKTCalculationStrategy {

    @Override

    public double hitungTagihan(int totalSks) {

        return 5000000 + (totalSks * 250000);

    }

}

```

Implementasi kalkulasi tarif kuliah bagi mahasiswa jalur masuk reguler.

```

package strategy;

public class InternationalUKTStrategy implements UKTCalculationStrategy
{

    @Override

    public double hitungTagihan(int totalSks) {

        return 15000000 + (totalSks * 500000);

    }

}

```

Implementasi tarif kuliah bagi kelas internasional dengan komponen biaya premium.

```

package strategy;

```

```
public class MbkmUKTStrategy implements UKTCalculationStrategy {

    @Override

    public double hitungTagihan(int totalSks) {

        return 2500000;

    }

}
```

Skema strategi baru untuk program Kampus Merdeka, ditambahkan secara eksternal tanpa mengusik sistem lama.

- Package repository

```
package repository;

import java.util.List;

public interface KRSDataRepository {

    void simpanKRS(String mahasiswaId, List<matakuliah.MataKuliah>
daftarMatkul);

}
```

Kontrak abstraksi data access layer yang memutus ketergantungan langsung terhadap vendor database (DIP).

```
package repository;

import java.util.List;

public class CloudNoSQLKRSRepository implements KRSDataRepository {

    @Override

        public void simpanKRS(String mahasiswaId,
List<matakuliah.MataKuliah> daftarMatkul) {

        System.out.println("Menyimpan data KRS ke Cloud NoSQL dengan
skalabilitas tinggi.");

    }

}
```

Detail implementasi baru berbasis NoSQL Cloud guna mengatasi beban puncak saat war KRS berlangsung.

```
package repository;

import java.util.List;
```

```

public class MySQLKRSRepository implements KRSDataRepository {

    @Override

        public void simpanKRS(String mahasiswaId,
List<matakuliah.MataKuliah> daftarMatkul) {

        System.out.println("Menyimpan data KRS ke MySQL Database secara
transaksional.");

    }

}

```

- Package KRS

```

package KRS;

public class KRSValidator {

    public boolean validasiPrasyarat(String mahasiswaId,
matakuliah.MataKuliah matkul) {

        System.out.println("Memvalidasi prasyarat akademik untuk: " +
matkul.getNama());

        return true;

    }

}

```

Komponen fungsional terisolasi untuk menangani aturan bisnis pemenuhan syarat pengambilan SKS (SRP).

```

package KRS;

import java.util.List;

public class KRSPdfGenerator {

    public void cetakDrafKRS(String mahasiswaId,
List<matakuliah.MataKuliah> daftarMatkul) {

        System.out.println("Menghasilkan file draf PDF KRS dengan
format kop surat resmi.");

    }

}

```

Komponen terisolasi untuk pembuatan draf PDF dokumen resmi mahasiswa (SRP).


```
package KRS;

import java.util.ArrayList;
import java.util.List;

import matakuliah.MataKuliah;
import repository.KRSDataRepository;
import strategy.UKTCalculationStrategy;

public class SistemKRSManager {

    private final KRSValidator validator;

    private final KRSPdfGenerator pdfGenerator;

    private final KRSDataRepository repository;

    public SistemKRSManager(KRSValidator validator, KRSPdfGenerator pdfGenerator, KRSDataRepository repository) {

        this.validator = validator;

        this.pdfGenerator = pdfGenerator;

        this.repository = repository;

    }

    public void prosesPendaftaranKRS(String mahasiswaId, List<MataKuliah> rencanaMatkul, UKTCalculationStrategy uktStrategy) {

        List<MataKuliah> matkulValid = new ArrayList<>();

        int totalSks = 0;

        for (MataKuliah matkul : rencanaMatkul) {

            if (validator.validasiPrasyarat(mahasiswaId, matkul)) {

                matkulValid.add(matkul);

                totalSks += matkul.getSks();

            }

        }

    }

}
```

```

        double totalTagihan = uktStrategy.hitungTagihan(totalSks);

        System.out.println("Total tagihan UKT terhitung: Rp" +
totalTagihan);

        repository.simpanKRS(mahasiswaId, matkulValid);

        pdfGenerator.cetakDrafKRS(mahasiswaId, matkulValid);

    }
}

```

Modul pengarah utama yang bersih dari logika detail. Bergantung sepenuhnya pada bentuk abstraksi repositori.

- Package main

```

package main;

import java.util.Arrays;
import java.util.List;

import KRS.*;
import matakuliah.*;
import repository.*;
import strategy.*;

public class App {

    public static void main(String[] args) {

        KRSValidator validator = new KRSValidator();

        KRSPdfGenerator pdfGenerator = new KRSPdfGenerator();

        KRSDataRepository repository = new CloudNoSQLKRSRepository();

        SistemKRSManager krsManager = new SistemKRSManager(validator,
pdfGenerator, repository);

        MataKuliah matkul1 = new MataKuliahTeori("MK01", "Rekayasa
Perangkat Lunak", 3);
    }
}

```

```

        MataKuliahPraktikum matkul2 = new MataKuliahPraktikum("MK02",
"Praktikum Jaringan Komputer", 1);

        List<MataKuliah> rencanaKrs = Arrays.asList(matkul1, matkul2);

        System.out.println("PROSES KRS MAHASISWA JALUR MBKM");

        UKTCalculationStrategy strategiMbkM = new MbkMUKTStrategy();

        krsManager.prosesPendaftaranKRS("MHS2024034", rencanaKrs,
strategiMbkM);

        System.out.println("\nEKSEKUSI OPERASI KHUSUS PRAKTIKUM");

        for (MataKuliah mk : rencanaKrs) {

            if (mk instanceof LaboratoriumAkses) {

                ((LaboratoriumAkses) mk).alokasiAsistenLab();

                ((LaboratoriumAkses) mk).cekPeralatanPraktikum();

            }

        }

    }

}

```

Titik mula program (entry point) yang diringkas menggunakan Wildcard Import untuk menyuntikkan dependensi secara manual.

5. Kesimpulan

Secara keseluruhan, rancangan ulang arsitektur sistem SIAKAD ini berhasil mengeliminasi kerapuhan kode monolitik lama melalui penerapan prinsip SOLID dan Decoupling Layer (DL) yang terorganisasi ke dalam struktur package berbasis lapisan logika. Dekomposisi kelas God Object SistemKRSManager menjadi komponen-komponen modular terisolasi berbasis Single Responsibility Principle (SRP) menjamin bahwa perubahan pada satu domain—seperti modifikasi draf PDF—tidak akan merusak fungsi persistensi data. Keberadaan Strategy Pattern lewat UKTCalculationStrategy memberikan fleksibilitas tinggi untuk mengakomodasi skema perhitungan baru seperti program MBKM tanpa mengutak-atik kode lama yang sudah stabil. Lebih lanjut, pemisahan kontrak abstrak KRSDataRepository dari kelas implementasi konkritnya (MySQLKRSRepository dan CloudNoSQLKRSRepository) memenuhi Dependency Inversion Principle (DIP), sehingga proses migrasi

infrastruktur database demi mengatasi beban puncak war KRS dapat dieksekusi secara instan dan aman tanpa mengganggu logika bisnis inti.

6. Daftar Pustaka

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). Modul Praktikum Pemrograman Berorientasi Objek: Java Collections. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.